

Automated Recipe Labeling Using A Bayesian Model

Buss, Sarah

Department of Computer Science, BYU

sgc255@byu.edu

Daniels, Cody

Department of Computer Science, BYU

codymd@byu.edu

Perry, Gabriel

Department of Computer Science, BYU

gmp99@byu.edu

June 13, 2026

Abstract

Our aim is to provide automated labeling for recipes based on the ingredients in a given recipe. We use a naïve Bayesian estimator to predict the output class of out-of-sample recipes with about 70% precision, surpassing our other attempts by no small margin. We also ran our model on the recipes having unique output classes to see how the model relabels these recipes.

1 Introduction

Our aim is to provide automated labeling for recipes based on the ingredients in a given recipe. Such labeling might help to clean existing databases, especially since many recipe labels are overly specific and unique to a single recipe. For example, 'Manicotti' or 'Beef Organ Meats' were both overly specific labels that we found in our data set. Such recipes could be more easily found if grouped with more general labels, such as "Pastas" or "Meat." This recipe labeling process would also be useful as a demonstration that similar labeling problems are solvable within a certain accuracy, such as genres of music, spoken or written language, and subjects or themes in writing.

2 The Data

We retrieved the data for this model from a Kaggle repository (1). It is open source, updated yearly, and sourced from the public site Food.com. We retrieved it using the secure Python API of Kaggle.

The data set includes about 522,510 entries consisting of the following features: RecipeId, Name, AuthorId, AuthorName, CookTime, PrepTime, TotalTime, DatePublished, Description, Images, Keywords, RecipeIngredientQuantities, RecipeIngredientParts, AggregatedRating, ReviewCount, Calories, FatContent, SaturatedFatContent, CholesterolCon-

tent, SodiumContent, CarbohydrateContent, FiberContent, SugarContent, ProteinContent, RecipeServings, RecipeYield, and RecipeInstructions.

We immediately removed many of the irrelevant features, such as IDs and image links. We initially kept the nutritional information, such as the cholesterol, fat, sugar, and sodium content, but after some testing, we determined that they would not be as useful in determining recipe category as the ingredients themselves. In total, we kept only two of the original features: RecipeIngredientParts and RecipeCategory (the output class). The RecipeIngredientParts feature from the Kaggle repository is given as a list of strings, which we use to count the frequency of the ingredients across different categories.

Table 1: An Example Data Point (raw from repository, dropped irrelevant features)

Feature	Value
RecipeIngredientParts	"plain tomato juice", "cabbage", "onion", ...
RecipeCategory (Output)	Vegetable

To simplify the output, we significantly reduced the number of output classes. We considered only the 40 most common labels and combined several similar labels into larger classes. After consolidation, we ended with **6 different output classes** (food genres) and this reduced our total instances to **272,913**, as many of the labels were singletons (only used for a single recipe) or similarly rare. Our final output classes were 'Dessert', 'Meat', 'Vegetable', 'Lunch/Snacks', 'Breads', and 'Breakfast'.

3 Initial Results

We first ran a simple decision tree on the presence of ingredients to get a feel for how categorical data might predict the output class. The model fit terribly. We were disappointed to see that the R^2 scores were above 6% on our test set for only a few hyper-parameter combinations.

We also tried a principal component analysis (PCA) and a decision tree on numeric features (nutrition facts) to determine which variables were the most important in discriminating output classes. These models also did very poorly. We then tried several approaches, including a new decision tree, a multilayer perceptron with one-hot encoding of the output class, and a k-nearest-neighbor model as well. All three of these models returned hopeful results on a subset of 1,000 recipes.

4 Final Model and Results

By far our best model was a naïve Bayesian estimator. We were able to predict the output class of out-of-sample recipes with about 70% accuracy, surpassing the other models by no small margin.

Our model predicts the probability vector over all classes/categories $P(category|ingredients)$

via the following computations:

$$P(\text{category}|\text{ingredients}) = \alpha P(\text{ingredients}|\text{category})P(\text{category}) \quad (1)$$

$$P(\text{ingredients}|\text{category}) = \prod_{\text{ingredients}} P(\text{ingredient}|\text{category}) \quad (2)$$

where we derive the needed quantities $P(\text{category})$ by counting the total recipes in each category (stored in a Python dict as $\text{priors}[\text{category}]$) and $P(\text{ingredient}|\text{category})$ by counting the instances of each ingredient for each category (also stored as a Python dict as $\text{ingredient_given_category}[\text{category}][\text{ingredient}]$). α is a normalization factor.

We then divide the ingredient counts by the total instances in each class/category to obtain their relative frequencies in each class. Ingredients not found in a class are given a default frequency of $1/(N + 1)$, where N is the size of the category (we assume that the next sampled recipe from that class would have had that ingredient; this is a very conservative estimation for novel ingredients, and using $N =$ the size of the entire training set has a similar out-of-sample performance).

We also ran our model on the recipes having unique output classes to see how the model relabels these recipes. Some examples of its choices are listed here:

	RecipeIngredientParts	RecipeCategory	prediction
522489	[onion, tomatoes, green bell pepper, green chi...	Indian	Lunch/Snacks
522490	[rolled oats, sweetened flaked coconut, sea sa...	Canadian	Dessert
522492	[ground beef, ground lamb, tomato paste, brand...	Australian	Meat
522493	[olive oil, water, low sodium soy sauce, lemon...	High Protein	Meat
522499	[sugar, vanilla extract, vanilla vodka]	Beverages	Dessert
522505	[green chilies, cheddar cheese, eggs, half & h...	Mexican	Breakfast
522509	[lemon wedge, granulated sugar, cognac, brandy...	Beverages	Dessert
522513	[Dijon mustard, garlic, peppercorns, shallot, ...	Very Low Carbs	Meat
522514	[half-and-half, heavy cream, brandy, sugar]	Ice Cream	Dessert
522515	[wasabi paste, dill, English cucumber, smoked ...	Canadian	Lunch/Snacks

Figure 1: Actual Rare Recipe Categories vs. Predicted Categories

As seen in Fig. 1, the relabeling often makes sense. For example, the second recipe from the top labeled Canadian contains sweetened coconut flakes and is relabeled as a dessert; the Mexican dish containing egg, cheese and chilies is relabeled as a breakfast; and the Australian dish features two types of meat, so it is relabeled as a meat dish.

5 Future Work

Expanding the scope of the categorization could work well. We could use more labels or apply a softmax to the predictions to accommodate multiple labels in recipes that fit into multiple categories. Our code runs regardless of the names and number of recipe classes, so including more classes should be simply a matter of feeding the algorithm the proper

table. We could also attempt to use the relative quantities of ingredients to produce a weighted or relative importance; for example, if a recipe contains only sugar and flour, the relative amount of each could determine whether the result is a dessert or a bread. We attempted this in our early iterations, but the current Kaggle data set does not properly scrape ingredient quantities, so we had to drop them. Another approach that might be promising is using a matrix partitioning algorithm to group recipes into blocks based on similarities in their ingredient lists. This approach could be useful for suggesting similar recipes or recipes using a certain set of available ingredients.

Acknowledgements

The authors thank Dr. Martinez and his TAs for the opportunity to investigate this topic as part of their Intro to Machine Learning course at Brigham Young University.

References

- [1] Alvin. (2020). Food.com - Recipes and Reviews, Version 1. Retrieved Oct 27 2023 from <https://www.kaggle.com/datasets/irkaal/foodcom-recipes-and-reviews/data>
- [2] Sarah Buss, Cody Daniels, and Gabriel Perry. "Recipes.ipynb". Google Colab Jupyter Notebook. <https://colab.research.google.com/drive/1Y3cTu-Afyr1S4L89oKY82RsqfdN1R93V?usp=sharing>

Appendix

We met about every other Friday during the second half of the semester to report our findings and had a group text to facilitate out-of-meeting communication. We planned with the deadline in mind and assigned tasks before dismissing. We divided the assignment roughly as follows:

- Propose the Project (Sarah)
- Select Data Set (Sarah)
- Download the Data (Gabriel)
- Initial Feature Selection (shared)
- Build and Test Initial Models (shared)
- Refined Feature Selection (shared)
- Output Class Selection and Consolidation (Cody)
- Bayes Model (Gabriel)
- Finalize Jupyter Notebook (2) (shared)
- Write the paper (shared)
- Prepare slides for oral presentation (shared)